

Artificial Neural Networks for Undergraduates

Semester B, 2026

General Instructions:

- The deadline for submission is June 10th at 23:59.
- Submission in pairs (State both students ID numbers).
- Each submission in the course Moodle must include:
 - Written PDF report (5-10 pages)
 - A github of the project (the link should be open access and written at the beginning of the PDF report)

Introduction:

In this project you will investigate how neural network architecture affects learning, performance, interpretability, and generalization.

You will work with the CIFAR-10 image dataset which contains 60,000 color images ($32 \times 32 \times 3$) from 10 classes: 1) airplane 2) automobile 3) bird 4) cat 5) deer 6) dog 7) frog 8) horse 9) ship 10) truck. The size of the training set will be of 50,000 images and the test set 10,000 images.

In the project you will perform 5 tasks. For each task you need to upload a different file/script into a project GitHub. Additionally, you need to write an explanation about your answer with supporting figures and visualizations to a PDF report. Grades will be given based on accuracy, clear reasoning, and concise explanations.

You are given with initial functions in the PDF "Initial functions" which allows you to load the data, have running initial models for training and testing etc. Most of your computational work is to improve specific parts and to add visualizations as required.

1. CNN debugging (25 points)

You are given initial code for a classifying CNN (called InitialCNN). Run the model and evaluate it. Your evaluation should report the following parameters: training loss, training accuracy and test accuracy. Additionally, plot the learning curves across epochs and another subset of plots which represent the size of the gradient magnitudes of each layer within each epoch during training. Hint- the standard training steps: after the forward pass, compute the loss, perform backpropagation to compute gradients, and update the model weights. In PyTorch, gradients can be accessed through `param.grad` after backpropagation.

Following the evaluation improve the architecture and retrain. Your improvement should be minor and include only one additional layer. Compare the results before and after you fix the code. Please note that the improvement should focus on basic components of CNN as were taught in class (like activation functions, normalization, architectural depth, etc) and not on additional expansions of the basic material.

2. Comparison to MLP (20 points)

Implement a Multi-Layer Perceptron that classifies CIFAR-10 images (Note to flatten each image into a vector). Use at least 2 hidden layers, train and evaluate it. Compare your results to the CNN in relation to accuracy, parameters (number of parameters and fine tuning of hyper parameters), training speed and size of memory required. Additionally, compare the differences in interpretability between the CNN and the MLP by visualizing the optimized weights of each architecture. Explain to which type of tasks each architecture is better.

3. Overfitting Analysis (20 points)

Train a larger version of one model long enough to create overfitting. Plot the train loss, validation loss, train accuracy as function of epochs and validation accuracy. Identify when the overfitting begins.

To avoid overfitting, compare two possible solutions, and illustrate how they overcome the problem:

- Dropout
- Weight decay
- Early stopping
- Smaller model

4. Augmentation (20 points)

In many image classifiers, it is common to use basic augmentations of the data including image transformations of random crop, rotation, zoom/resize and horizontal/vertical flips.

Perform these transformations on the data to create additional images, and demonstrate for each transformation how it affected the image.

Now retrain the CNN using the augmented data and compare the test accuracy with and without augmentation. Note to use figures to support your claims (for example by showing confusion matrix analysis etc.).

5. Creativity task (15 points)

Design your own hybrid network using both CNN and MLP ideas.

Examples:

- CNN feature extractor + MLP classifier
- MLP branch + CNN branch
- Patch-based MLP + CNN fusion

In your report add a draw of your new architecture, explain the intuition behind it and how it should work, implement the model and compare it to the standard CNN and MLP you used.